How to *Not* Mess Up
PowerShell for Security with AI



Yossi Sassi

HACKTIVITY

```
Internet Reset
Global Cache Cleared

F:\>
```

WhoAmI





- InfoSec Researcher; friendly H@ↄk3r
- Red mind, Blue heart
- Co-Founder @ TENROOT; Advisory @ terra, TandemTrace, RAVEN
- 35+ years of keyboard access – Code, IT Security, Network Protocols, DF/IR
- Ex-Javelin Networks (Acquired by Symantec)
- Ex-Technology Group Manager @ Microsoft
- Volunteer (Youth at risk); Oriental Rock Bouzoukitarist; Pilot


Perl
C#
BASH
sh

Vibe coders these days



All you need is… mmm… Good taste? ☺

- We've all been there:
  Prompt for a PowerShell script.

  Script looks right.

  Indentation – flawless.

  Helpful comments, sensible structure.

- … Yet wayyy too many lines (long & unclear)
- …. and somewhere in the middle there'll be a command that never existed, or a parameter that was retired years ago.
- … And other issues (errors at runtime / **unsafe to run**!)



PowerShell failures are often quiet.

- Under PowerShell's default settings, a filter that refers to *a property which does not exist* will often **match nothing** rather than raise an error.

- **Commands behavior & availability differ between Windows PowerShell 5.1 and PowerShell 7.x** (can do exactly the wrong thing, without an error), as well as in module compatibility and the underlying .NET runtime.

- An *-ErrorAction SilentlyContinue* added to keep the output clean will **suppress precisely the failure you needed to see**.


Looks right
!=
Is right

Why is it like THAT with PowerShell & AI?

- Much of the “best” PowerShell was never published.
- a great deal of the most valuable **production administration code** never leaves the organization.
  - Tenant migration tooling, Active Directory remediation, monitoring libraries, deployment scripts etc. sit inside managed service providers, internal IT departments and consultancies.
- What remains publicly visible tends to be more ‘one-off snippets’, forum answers and blog posts.

Why? (continued): **A decade+ of Changes**

- PowerShell is a fairly young language, keeping up between OnPrem to .NET efforts to Cloud efforts to OS efforts to AI.

- Packages/Modules change and/or decommission all the time
  - e.g. MSOnline and AzureAD modules deprecated in favor of the Microsoft Graph PowerShell SDK

Why? (Continued)**: Testing PowerShell is hard**

- AI can run generated code automatically and reward the model when it works. Easy, right?

- Pretty straightforward for Python: spin up a container, run the tests, score the result, repeat at enormous scale.
- PowerShell 7.x runs perfectly well on Linux, so the language can be executed in an ordinary container.

- The difficulty is reproducing **the environments** in which useful PowerShell *actually operates*.

It used to be worse. It's STILL BAD.

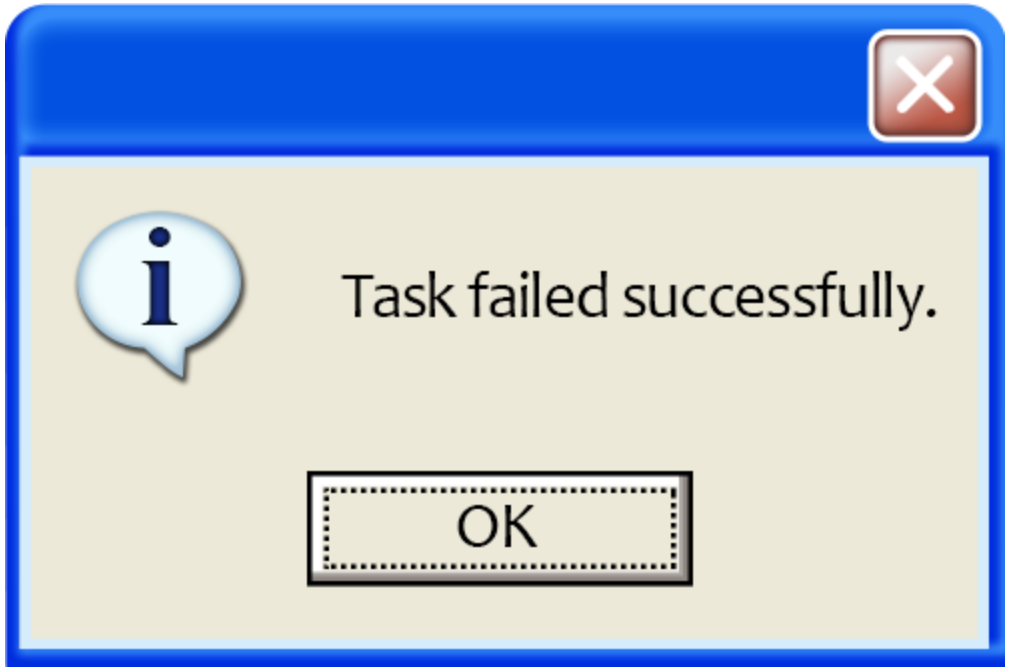
- Solution? AI assistants may use **Strict Mode**, or try to tackle it by **reverting to C#** ☺

- a January 2026 research introduced **SecGenEval-PS** to assess PowerShell generation, security analysis and repair, and found that more than **60%** of the scripts produced by GPT were insecure without structured guidance.

- Bottom line: output looks great, code is **not safe to run**.

Your AI-Generated PowerShell Script

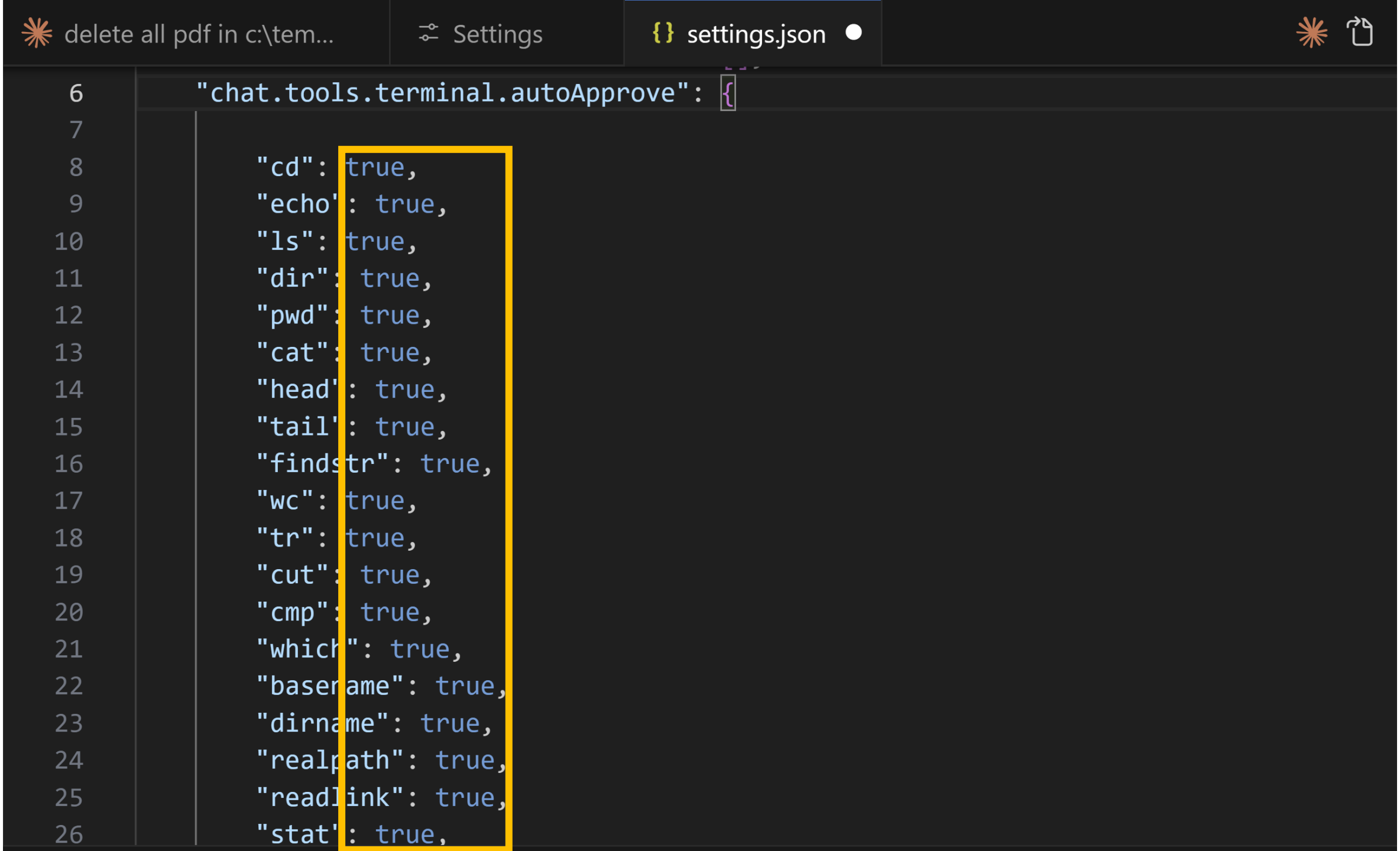



Task failed successfully.
OK

Local agents can lose control easily

Example: **Auto-approved terminal commands/tools**

```
  6    "chat.tools.terminal.autoApprove": {
110        "spps": false,
111        "taskkill": false,
112        "taskkill.exe": false,
113        "curl": false,
114        "wget": false,
115        "Invoke-RestMethod": false,
116        "Invoke-WebRequest": false,
117        "irm": false,
118        "iwr": false,
119        "chmod": false,
120        "chown": false,
121        "Set-ItemProperty": false,
122        "sp": false,
123        "Set-Acl": false,
124        "jq": false,
125        "xargs": false,
126        "eval": false,
127        "Invoke-Expression": false,
128        "iex": false
129    }
```


delete all pdf in c:\tem...
Settings
settings.json
"chat.tools.terminal.autoApprove": {
"cd": true,
"echo": true,
"ls": true,
"dir": true,
"pwd": true,
"cat": true,
"head": true,
"tail": true,
"findstr": true,
"wc": true,
"tr": true,
"cut": true,
"cmp": true,
"which": true,
"basename": true,
"dirname": true,
"realpath": true,
"readlink": true,
"stat": true,

delete all pdf in c:\tem... | Settings | settings.json ●

```
"chat.tools.terminal.autoApprove": {

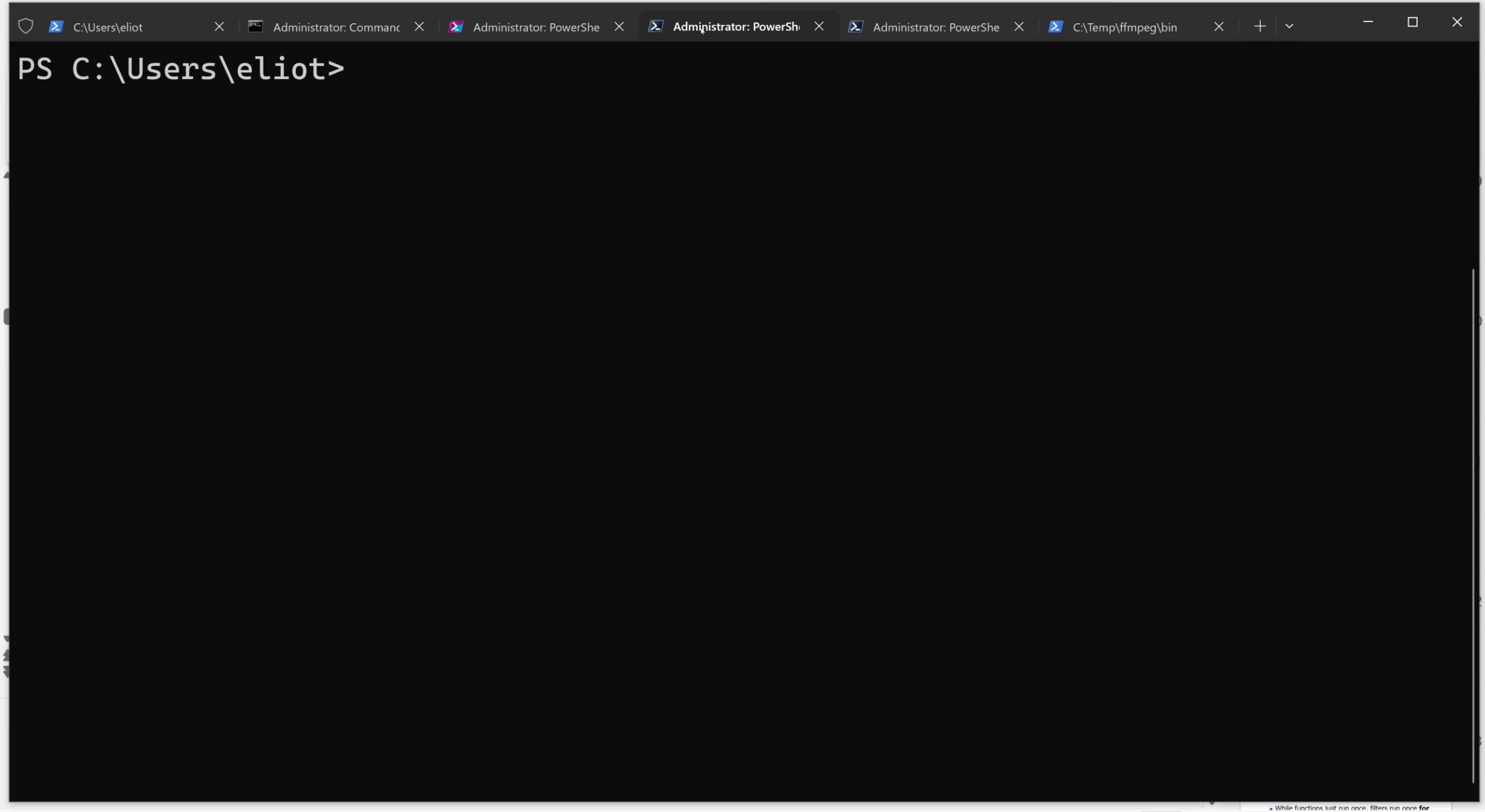
    "cd": true,
    "echo": true,
    "ls": true,
    "dir": true,
    "pwd": true,
    "cat": true,
    "head": true,
    "tail": true,
    "findstr": true,
    "wc": true,
    "tr": true,
    "cut": true,
    "cmp": true,
    "which": true,
    "basename": true,
    "dirname": true,
    "realpath": true,
    "readlink": true,
    "stat": true,
```



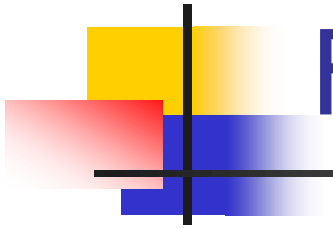
Turning Low-Risk Commands Into Payloads

```
echo calc.exe > .\payload.txt
$cmd = cat .\payload.txt
& $cmd
```

- echo, cat etc. are normally considered low-risk.
- The risk emerges when data is read from a file, registry value, network response, or other source is later treated as executable code (&, script blocks, dynamic loading, etc.).
- **Individual commands = Secure; Sequence = Nope** ☺


C:\Users\eliot
Administrator: Commanc
Administrator: PowerShe
Administrator: PowerSh
Administrator: PowerShe
C:\Temp\ffmpeg\bin
PS C:\Users\eliot>

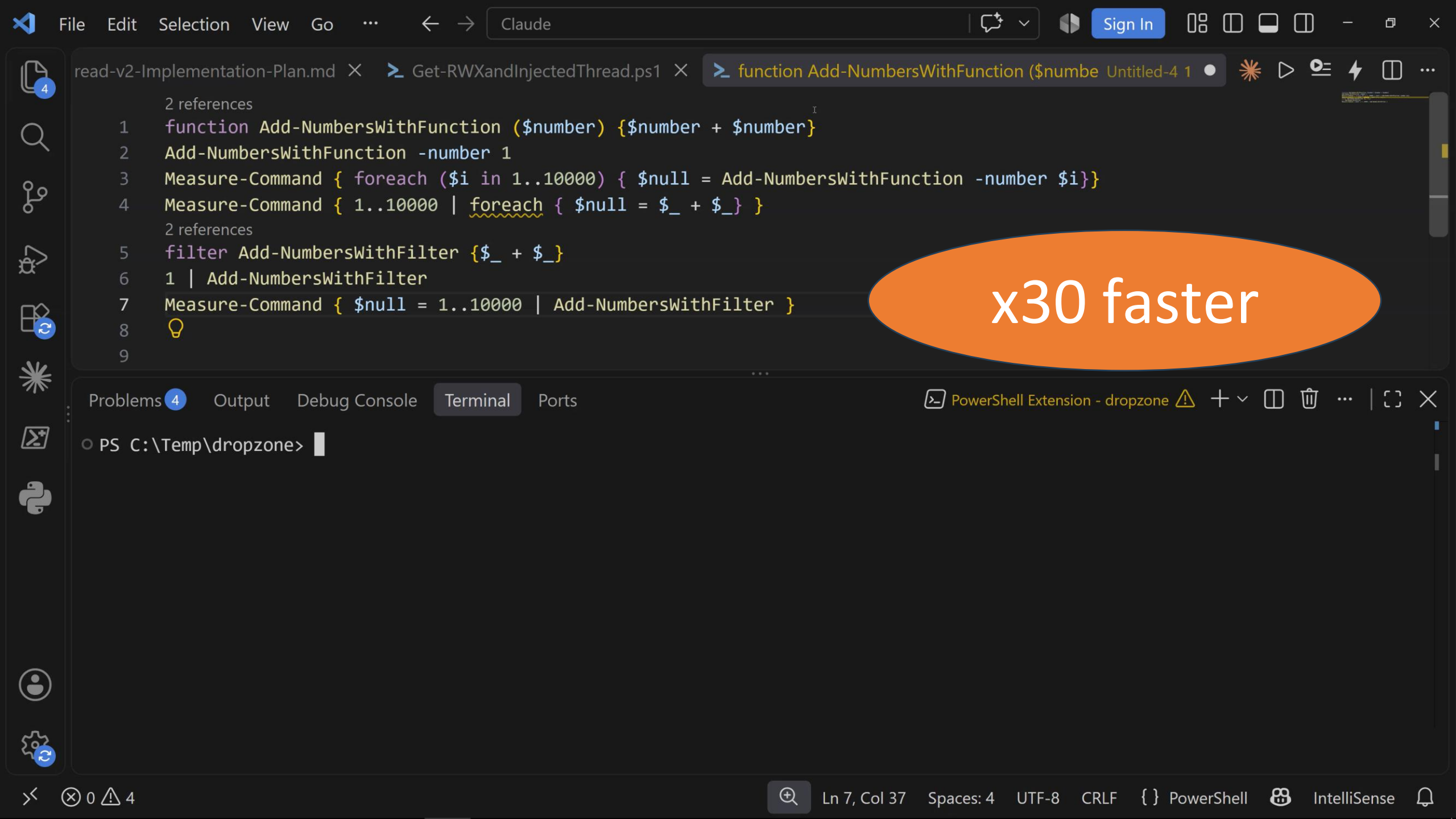
AI doesn't care about
Correct | Secure | efficient PowerShell Code



Filters

- Similar to a function
- Includes a ‘built-in’ **foreach**

- While functions just run once, filters run once **for each input object in the pipeline**


File Edit Selection View Go
Claude
Sign In
read-v2-Implementation-Plan.md
Get-RWXandInjectedThread.ps1
function Add-NumbersWithFunction ($numbe
Untitled-4 1
2 references
function Add-NumbersWithFunction ($number) {$number + $number}
Add-NumbersWithFunction -number 1
Measure-Command { foreach ($i in 1..10000) { $null = Add-NumbersWithFunction -number $i}}
Measure-Command { 1..10000 | foreach { $null = $_ + $_} }
2 references
filter Add-NumbersWithFilter {$_ + $_}
1 | Add-NumbersWithFilter
Measure-Command { $null = 1..10000 | Add-NumbersWithFilter }
x30 faster
Problems 4
Output
Debug Console
Terminal
Ports
PowerShell Extension - dropzone
PS C:\Temp\dropzone>
0 4
Ln 7, Col 37
Spaces: 4
UTF-8
CRLF
PowerShell
IntelliSense

Performance Example:
Process Owner

```
PS ► Measure-Command{$LoggedOnUser = Get-WmiObject -Class Win32_Process -Filter "Name='explorer.exe'" |
>>                ForEach-Object {
>>                  $owner = $_.GetOwner();
>>                  "$($owner.Domain)\$($owner.User)"
>>                } | Select-Object -Unique}


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 233
Ticks             : 2337866
TotalDays         : 2.70586342592593E-06
TotalHours        : 6.494072222222222E-05
TotalMinutes      : 0.00389644333333333
TotalSeconds      : 0.2337866
TotalMilliseconds : 233.7866



PS ► Measure-Command{(Get-Process -Name explorer -ErrorAction SilentlyContinue -IncludeUserName).username}


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 8
Ticks             : 89078
```

x30 faster

Example: Registry

```
PS ► measure-command {Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion" |
>> Select-Object ProductName, EditionID, ReleaseId, DisplayVersion, CurrentBuild
>> }

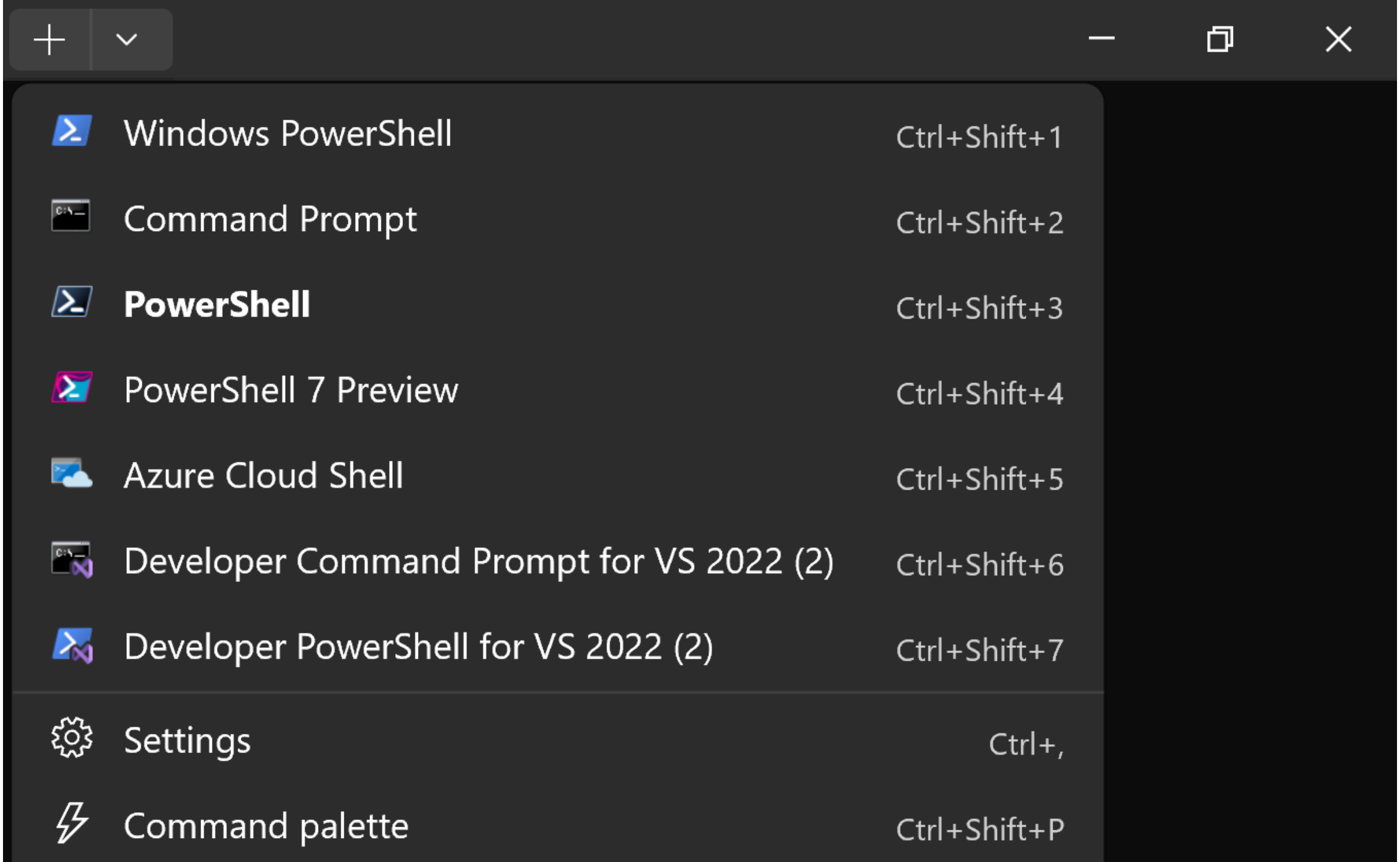

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 2
Ticks             : 23102
TotalDays         : 2.67384259259259E-08
TotalHours        : 6.41722222222222E-07
TotalMinutes      : 3.85033333333333E-05
TotalSeconds      : 0.0023102
TotalMilliseconds : 2.3102



PS ► measure-command {Get-CimInstance Win32_OperatingSystem | Select-Object Caption, Version, BuildNumber, OSArchi
tecture
>> }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 102
```

x50 faster!


Windows PowerShell Ctrl+Shift+1
Command Prompt Ctrl+Shift+2
PowerShell Ctrl+Shift+3
PowerShell 7 Preview Ctrl+Shift+4
Azure Cloud Shell Ctrl+Shift+5
Developer Command Prompt for VS 2022 (2) Ctrl+Shift+6
Developer PowerShell for VS 2022 (2) Ctrl+Shift+7
Settings Ctrl+,
Command palette Ctrl+Shift+P

```
PowerShell host: 64-bit. Target process: WUDFHost (PID 1884)
Ensure you run matching bitness for 64-bit targets.
Exported JSON to C:\temp\suspicious_threads1.json
No suspicious exec+write or exec-only private committed regions found.

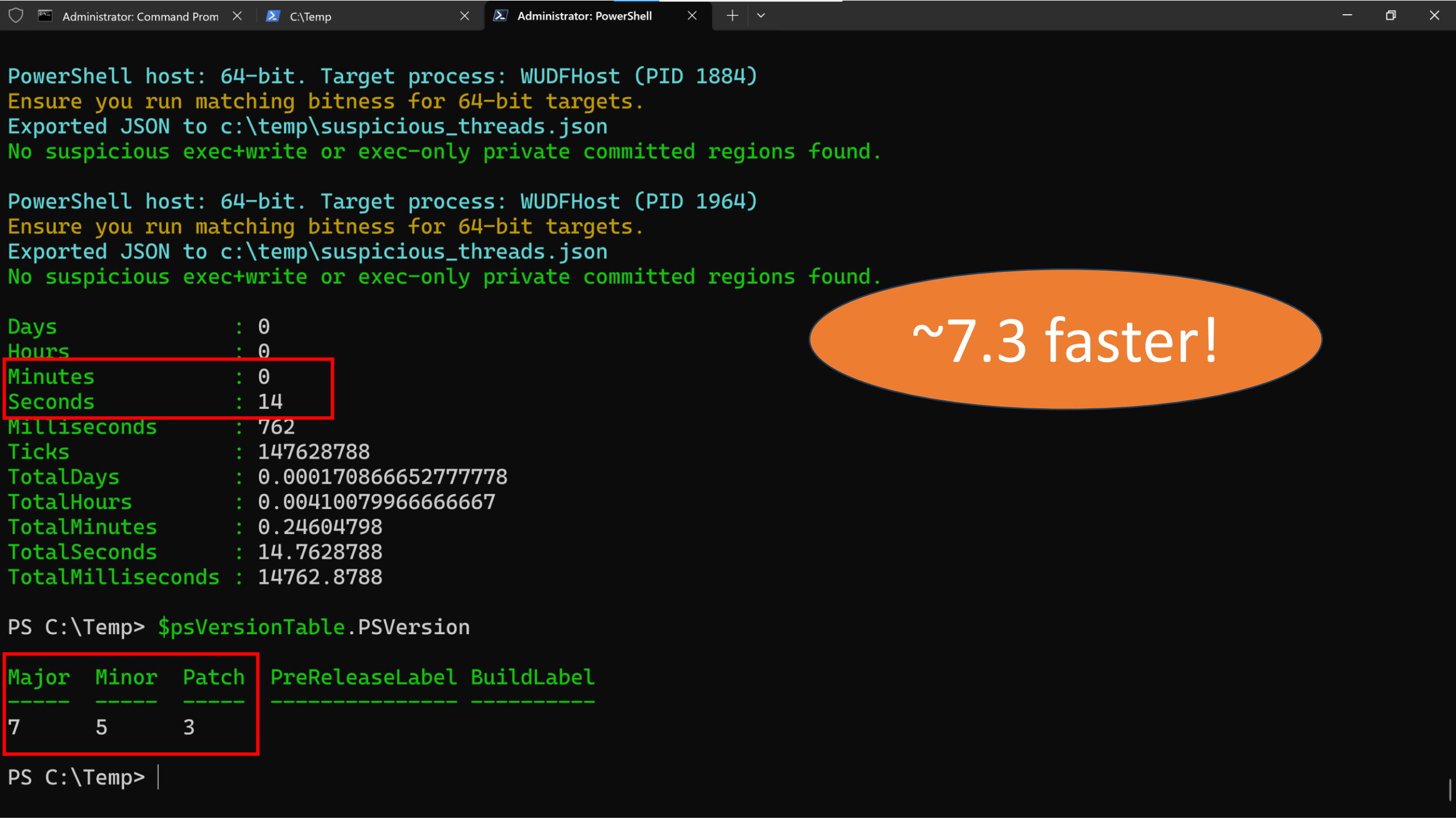
PowerShell host: 64-bit. Target process: WUDFHost (PID 1964)
Ensure you run matching bitness for 64-bit targets.
Exported JSON to C:\temp\suspicious_threads1.json
No suspicious exec+write or exec-only private committed regions found.


Days              : 0
Hours             : 0
Minutes           : 1
Seconds           : 42
Milliseconds      : 669
Ticks             : 1026693665
TotalDays         : 0.00118830285300926
TotalHours        : 0.0285192684722222
TotalMinutes      : 1.71115610833333
TotalSeconds      : 102.6693665
TotalMilliseconds : 102669.3665



PS ► $psVersionTable.PSVersion

Major  Minor  Build  Revision
-----  -----  -----  --------
5      1      19041  6328
```


Administrator: Command Prom
C:\Temp
Administrator: PowerShell
PowerShell host: 64-bit. Target process: WUDFHost (PID 1884)
Ensure you run matching bitness for 64-bit targets.
Exported JSON to c:\temp\suspicious_threads.json
No suspicious exec+write or exec-only private committed regions found.
PowerShell host: 64-bit. Target process: WUDFHost (PID 1964)
Ensure you run matching bitness for 64-bit targets.
Exported JSON to c:\temp\suspicious_threads.json
No suspicious exec+write or exec-only private committed regions found.
Days : 0
Hours : 0
Minutes : 0
Seconds : 14
Milliseconds : 762
Ticks : 147628788
TotalDays : 0.0001708666652777778
TotalHours : 0.004100799966666667
TotalMinutes : 0.24604798
TotalSeconds : 14.7628788
TotalMilliseconds : 14762.8788
PS C:\Temp> $psVersionTable.PSVersion
Major Minor Patch PreReleaseLabel BuildLabel
7 5 3
PS C:\Temp>
~7.3 faster!

Windows PowerShell != PowerShell core





… and AI will mix them up.

Windows PowerShell ! = PowerShell core

Feature	Windows PowerShell	PowerShell Core
Naming	PowerShell	pwsh, or Microsoft PowerShell
Platform Support	Windows only	Cross-platform Win/Mac/Linux
Version	Up to 5.1	6.0 and later (7.x)
Framework	.NET Framework	.NET Core / .NET 5+
Compatibility	Best with Windows-specific modules and cmdlets	Cross-platform compatibility, subset of .NET Framework features
Installation	Pre-installed on Windows	Separate installation required; download from GitHub / winget
Performance	Usually slow (2-phase compilation)	Somewhat faster (Parallel %)*
Audit/Transcription	via GPO/Registry	N/A *

Windows PowerShell = !PowerShell core (continued)

- Multiple syntax inconsistencies exist between v5.1 to v7.x
  - e.g. **Set-Content** for **bytes** (exists in Windows PowerShell only)

- This will work on *both editions* (Core + Windows):

  [IO.File]::WriteAllBytes($OutputFile,$Bytes)

Windows PowerShell != PowerShell core

Best practice: Guide AI to use a specific Edition & Version (may still produce ‘bad’ code…),

- or -

Guide AI to use generic .NET rather than built-in cmdlets

Starting a new process – not always trivial

```
PS ► docker-compose exec -T database psql -U postgres < supabase/migrations/0001_si
mple_setup.sql
```

This will not work,
of course…

```
PS ► start cmd -ArgumentList 'docker-compose exec -T database psql -U postgres < supaba
se/migrations/0001_simple_setup.sql'
```

Proper way + can extend control
with PassThru & Redirect Stdin/out

titled

powershell to get netstat -ano output and get process information (path, manufacturer, signature etc) using the PID from netstat, and get the remote ip information using invoke-webrequest https://ipinfo.io/$_/json | convertfrom-json . append remote ip info and process info to the netstat entry. save objects into a global variable

ll create a PowerShell script that combines netstat output with process information nd remote IP geolocation data.

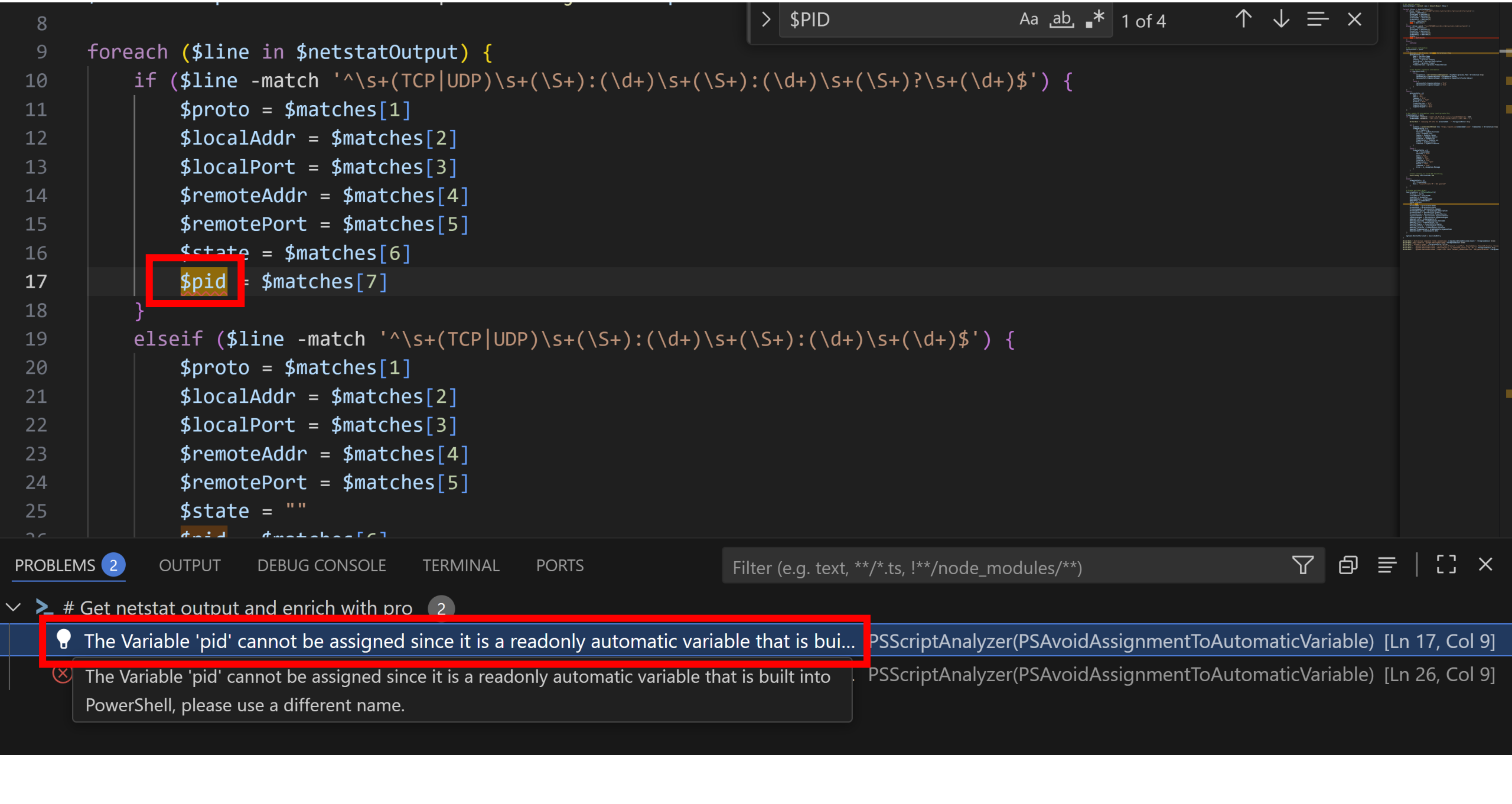
Network Connections Analysis Script

Code



```
Cannot overwrite variable PID because it is read-only or constant.
At line:9 char:9
+         $pid = $matches[7]
+         ~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : WriteError: (PID:String) [], SessionStateUnauthorizedAccessException
    + FullyQualifiedErrorId : VariableNotWritable

Cannot overwrite variable PID because it is read-only or constant.
At line:9 char:9
+         $pid = $matches[7]
+         ~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : WriteError: (PID:String) [], SessionStateUnauthorizedAccessException
    + FullyQualifiedErrorId : VariableNotWritable
```


$PID
Aa ab .* 1 of 4
foreach ($line in $netstatOutput) {
if ($line -match '^\s+(TCP|UDP)\s+(\S+):(\d+)\s+(\S+):(\d+)\s+(\S+)?\s+(\d+)$') {
$proto = $matches[1]
$localAddr = $matches[2]
$localPort = $matches[3]
$remoteAddr = $matches[4]
$remotePort = $matches[5]
$state = $matches[6]
$pid = $matches[7]
}
elseif ($line -match '^\s+(TCP|UDP)\s+(\S+):(\d+)\s+(\S+):(\d+)\s+(\d+)$') {
$proto = $matches[1]
$localAddr = $matches[2]
$localPort = $matches[3]
$remoteAddr = $matches[4]
$remotePort = $matches[5]
$state = ""
PROBLEMS 2
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Filter (e.g. text, **/*.ts, !**/node_modules/**)
Get netstat output and enrich with pro 2
The Variable 'pid' cannot be assigned since it is a readonly automatic variable that is bui... PSScriptAnalyzer(PSAvoidAssignmentToAutomaticVariable) [Ln 17, Col 9]
The Variable 'pid' cannot be assigned since it is a readonly automatic variable that is built into PowerShell, please use a different name.
PSScriptAnalyzer(PSAvoidAssignmentToAutomaticVariable) [Ln 26, Col 9]

$PID is an automatic, read-only variable in PowerShell.

But I wrote ‘PID’ so AI *took that word **literally***
as the variable name –
overlooking the fact that this creates an obvious bug!

Working Well with AI & PowerShell

- Ask AI for a **one-liner**, when task requires **ad-hoc functionality**.

Alternative one-liner if you just want the name:

powershell

```
(Get-ADDomainController -Filter * | ForEach-Object { [PSCustomObject]
```

Copy

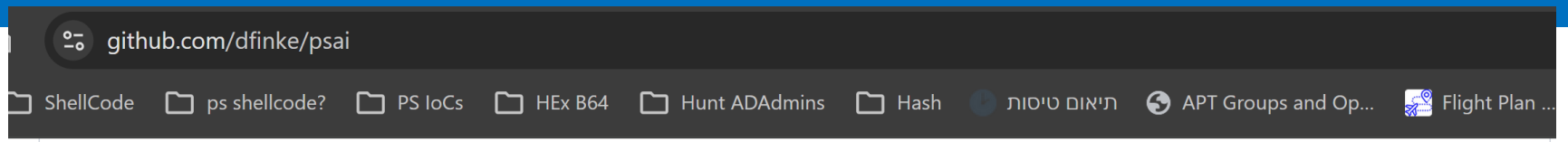
(Get-ADDomainController -Filter * | ForEach-Object { [PSCustomObject]@{Name=$_.Name; Created=(Get-ADComputer $_.Name -Properties whenCreated).whenCreated} } | Sort-Object Created | Select-Object -First 1).Name

Working Well with AI & PowerShell

- Ask AI for a **one-liner**, when task requires **ad-hoc functionality**.

- Prepare in advance **the practices you need** (skill file(s), custom instructions etc.) **to cover the known gaps**. (RAG?)

Convert Code Repositories to AI-optimized XML



Convert Your Repositories to AI-Ready Format

The `ConvertTo-AIPrompt` function packages any GitHub repository into an AI-optimized XML format, making it easy to feed your codebase to AI tools like ChatGPT, Claude, or Gemini.

Key Features

- **AI-Optimized**: Formats your codebase in a way that's easy for AI to understand and process.
- **Simple to Use**: You need just one command to pack your entire repository.
- **Customizable**: Easily configure what to include or exclude.

Basic Example

```
# Export a repository to a single file
ConvertTo-AIPrompt -RepoSlug "dfinke/ImportExcel" -OutputPath "D:\ImportExcel.txt"

# Include only specific file types
ConvertTo-AIPrompt -RepoSlug "dfinke/ImportExcel" -Include "*.ps1","*.md" -OutputPath "D:\ImportExcel-d

# Exclude specific file types
# Look at a single directory in the repo
ConvertTo-AIPrompt -RepoSlug "dfinke/ImportExcel/Examples" -Exclude "*.xlsx" -OutputPath "D:\ImportExce

# Return as string instead of saving to file
$repoContent = ConvertTo-AIPrompt -RepoSlug "username/repo"
```

Working Well with AI & PowerShell

- Ask AI for a **one-liner**, when task requires **ad-hoc functionality**.
- Prepare in advance **the practices you need** (skill file(s), custom instructions etc.) **to cover the known gaps**. (RAG?)
- Consider **Set-StrictMode -Version 2.0** for testing, so missing properties & uninitialized variables raise errors vs. passing silently.
- Run the finished script through **PSScriptAnalyzer**, Microsoft's static checker, which will flag uninitialized variables, compatibility problems and other practices before anything executes.
- **Your knowledge matters** – AI cannot tell your intended business logic, or how a remote service will behave (e.g. WinRM vs. RPC). Explain the architecture, Dos & Don'ts of what you need (*context*).

Key Things To Remember

- AI is fast & scalable, *not* necessarily smart.
  - **Less coding = More explaining / Controls**

Guardrails

- **Security & Performance** don't matter to most LLMs, unless you guide them = Tell & Show them.

- **AI Doesn't "know"** Anything ☺ **<u>And Struggles with it</u>**.
  - Prepare practices in advance, check statically, understand the underlaying technology & business

Links / Thanks

- **Lightweight Yet Secure: Secure Scripting Language Generation via Lightweight LLMs** (SecGenEval-PS)
  - https://arxiv.org/abs/2601.06419

- **The Language AI Forgot**
  https://www.trichromic.co.uk/news/the-language-ai-forgot-why-powershell-trips-up-ai-assistants

- Fellow team @ TENROOT



1nTh35h3ll

YossiSassi

Red mind,Blue heart; Look.Think.Do; HAcktive Directory guy **@TenRoot**; Advisor/AI H@ck3r@ Terra, TandemTrace, Raven.io; People.Music.Code; Knowledge is PowerShell

Edit profile

294 followers · **4** following

TenRoot | Terra | TandemTrace | Raven.io

wherever I lay my IP

Pinned

Customize your pins

hAcKtive-Directory-Forensics Public

54 11

Token-Manipulation-Hunt Public

Detect token-manipulation behavior by correlating identity transitions, process access and resulting privileged process. Reconstructs the relationship between the process that accessed a privileged...

PowerShell

Get-UserSession Public

Query user sessions for the entire domain (Interactive/RDP etc), allowing you to query a Username and see all their logged on sessions, whether Active or Disconnected

Get-RemotePSSession Public

Query PS Sessions/WS-Man for their connected users, IPs, hosts, session times and more, for local and remote sessions, both Windows PowerShell and pwsh

Get-LogonCount Public

Count AD account(s) logons - Queries the logonCount attribute from all domain controllers (RWDC + RODC) for all or specific user accounts (and optionally computer accounts), with additional statist...

PowerShell 3 1

SEC-T_21-One-Liners-Powershell Public

Code & other materials from SEC-T 2022 talk "When SysAdmin & Hacker Unite: 21 One-Liners to make you convert from bash to Powershell"

PowerShell 16 3

Github.com/YossiSassi

172 contributions in the last year

Contribution settings

2026

2025

2024

Oct Nov Dec Jan Feb Mar Apr May Jun Jul Aug Sep Oct

Mon

Wed

köszönöm



Yossi_Sassi

yossis@protonmail.com

Enjoy HACKTIVITY!